Industr.IA

E-book

IA Generativa:

Foundation Models e panorâmica arquitetural



Curso base de IA | Unidade M1

Prof.: Arthur Nascimento

SUMÁRIO

1. Modelos discriminativos e modelos generativos 3
2. Panorâmica arquitetural em alto nível 7
3. Foundation Models e capacidade emergente 12
Conclusão 17

Unidade M1

1. Modelos discriminativos e modelos generativos

Esta seção pertence à camada conceitual e tem por objetivo que você distinga, com precisão, dois modos de modelar um problema de aprendizado de máquina: o modo discriminativo e o modo generativo. É a primeira peça do vocabulário desta lente, e a mais importante, porque separa a maior parte das soluções que você já encontrou no M0, construídas para decidir, daquelas que dão nome à IA Generativa, construídas para produzir. Sem essa distinção firme, toda decisão posterior do módulo fica sem chão.

No M0, um modelo foi descrito como uma função com valores ajustáveis que recebe uma entrada e devolve uma saída. Os exemplos daquele módulo compartilham um traço: o classificador de imagens, o modelo de preço de imóvel e a floresta aleatória do notebook comparativo recebem um exemplo e respondem com um rótulo ou um número. Todos são modelos discriminativos. Um modelo discriminativo aprende a fronteira entre as respostas possíveis: dado um exemplo, ele decide a que categoria pertence ou que valor assumir. Sem precisar da fórmula aqui, basta reter que ele aprende a relação entre a entrada e a resposta, e nada além disso. Não lhe interessa como os dados de entrada se distribuem no mundo, apenas onde fica a linha que separa uma resposta da outra.

Vale recuperar a imagem usada no M0, da reta que separa pontos azuis de pontos vermelhos num plano. Um modelo discriminativo é exatamente essa reta: ele aprende onde traçar a divisão para acertar de que lado fica cada ponto. Um modelo generativo faz algo diferente. Em vez de traçar a divisa, ele aprende como as nuvens de pontos azuis e vermelhos se distribuem, isto é, a forma de cada conjunto. E quem aprendeu a forma das nuvens pode fazer algo que a reta jamais faria: sortear um ponto azul novo, plausível, que não estava nos dados. Essa é a virada conceitual. O modelo generativo aprende a estrutura dos próprios dados, e essa estrutura aprendida é o que lhe permite produzir exemplos inéditos parecidos com os que viu.

Trocando o plano de pontos por dados reais, a mesma mecânica gera artefatos úteis. Um modelo generativo de texto, depois de modelar como frases plausíveis em português se encadeiam, escreve frases que não estavam no treino. Um modelo generativo de imagem, depois de modelar como fotografias se organizam, sintetiza imagens que nunca existiram. Um modelo generativo tabular, depois de modelar como as colunas de uma base se relacionam, produz linhas sintéticas coerentes com a base original. A diferença em relação ao modelo discriminativo não é de tamanho nem de modernidade, é de objetivo: discriminar é decidir entre opções dadas; gerar é produzir um artefato novo a partir da estrutura aprendida.

Há uma diferença de natureza no ato de produzir que convém explicitar. Quando um modelo discriminativo recebe uma entrada, sua resposta é, em geral, determinística: a mesma entrada leva ao mesmo rótulo. Quando um modelo generativo produz, ele amostra de uma distribuição que aprendeu, e por isso o mesmo pedido pode levar a artefatos diferentes a cada vez, todos plausíveis.

Essa variabilidade não é defeito, é consequência de o modelo ter aprendido um espaço de possibilidades, e não uma única resposta certa. Ela explica por que duas execuções de um gerador de texto sobre o mesmo tema rendem parágrafos distintos, e porque controlar essa variação, decidir quanta liberdade dar à geração, é uma preocupação prática da lente generativa, retomada quando se discute o uso responsável dessas ferramentas.

GLOSSÁRIO

Modelo discriminativo e modelo generativo

Modelo discriminativo é aquele que aprende a fronteira entre as respostas possíveis e, dada uma entrada, devolve um rótulo ou um valor. Modelo generativo é aquele que aprende a estrutura dos próprios dados e, por isso, pode produzir exemplos novos plausíveis. A diferença está no objetivo de modelagem, não no tamanho do modelo.

A forma mais curta de fixar a distinção é olhar para a pergunta que cada modelo responde. O modelo discriminativo responde a que classe pertence este exemplo, ou que número corresponde a esta entrada. O modelo generativo responde como se parece um exemplo plausível e, por extensão, produza um.

Essa diferença de pergunta determina o que se pode pedir ao modelo e, portanto, que tipo de problema cada um resolve bem. Classificar mensagens entre desejadas e indesejadas, decidir se uma peça apresenta defeito, prever a demanda de um produto na próxima semana ou ordenar candidatos por relevância são perguntas discriminativas, e um modelo discriminativo é o instrumento natural. Redigir o resumo de um relatório, escrever um trecho de código a partir de uma descrição, desenhar uma imagem a partir de um texto ou completar uma base com linhas sintéticas plausíveis são perguntas generativas.

O quadro a seguir reúne os dois modos lado a lado.

Quadro 1 - Modelo discriminativo versus modelo generativo.

Aspecto	Discriminativo	Generativo
O que aprende	a fronteira entre as respostas	a estrutura dos dados
A pesquisa que responde	a que classe ou valor corresponde a entrada	como se parece um exemplo plausível
O que produz	um rótulo ou um n[úmero	um artefato novo (texto, imagem, dado)
Custo típico	menor, inferência rápida	maior, inferência mais cara
Exemplos	floresta aleatória, regressão, SVM	modelos de linguagem, modelos de difusão

Para Cego Ver:

Quadro intitulado "Quadro 1- Modelo discriminativo versus modelo generativo". A imagem apresenta uma tabela comparativa com três colunas: Aspecto, Discriminativo e Generativo.

Na linha "O que aprende", o modelo discriminativo aprende a fronteira entre as respostas ou classes, enquanto o modelo generativo aprende a estrutura dos dados.

Na linha "A pergunta que responde", o modelo discriminativo busca determinar a qual classe ou valor uma entrada corresponde. Já o modelo generativo procura responder como seria um exemplo plausível gerado a partir dos padrões aprendidos.

Na linha "O que produz", o modelo discriminativo gera um rótulo ou valor numérico, enquanto o modelo generativo produz um novo artefato, como texto, imagem ou outro tipo de dado.

Na linha "Custo típico", o modelo discriminativo apresenta menor custo computacional e inferência mais rápida. O modelo generativo possui maior custo computacional e inferência mais onerosa.

Na última linha, "Exemplos", são citados como modelos discriminativos a floresta aleatória, a regressão e as máquinas de vetores de suporte (SVM). Como exemplos de modelos generativos aparecem modelos de linguagem e modelos de difusão.

Há um uso menos óbvio do modelo generativo que ajuda a entender o que ele de fato aprende. Como ele modela quão plausível é um exemplo, pode também medir o quanto um caso novo se afasta do que considera normal. Essa propriedade sustenta a detecção de anomalias: um modelo que aprendeu a estrutura do funcionamento normal de um equipamento atribui baixa plausibilidade a leituras atípicas, e isso sinaliza um possível defeito antes mesmo que alguém o rotule. O ponto conceitual é que produzir exemplos e avaliar plausibilidade são duas faces da mesma capacidade, a de ter modelado a estrutura dos dados, e não apenas a fronteira entre rótulos.

A distinção também aparece no tipo de dado de que cada modo precisa para aprender. Um modelo discriminativo costuma exigir pares rotulados, isto é, exemplos acompanhados da resposta correta, e rotular dados é caro e lento. Muitos modelos generativos, ao contrário, aprendem a estrutura dos dados a partir de material não rotulado, por aprendizado auto-supervisionado, em que o próprio dado fornece o sinal de treino, como ao prever a próxima palavra de um texto a partir das anteriores. Essa diferença tem peso prático: dados não rotulados são abundantes, e é em boa parte por isso que modelos generativos de grande escala puderam ser treinados sobre volumes que nenhum esforço de rotulagem manual alcançaria. Retomaremos esse ponto mais à frente, na seção que trata de Foundation Models.

Convém, desde já, evitar uma leitura rígida demais dessa fronteira. As duas categorias descrevem o objetivo de modelagem, não muros intransponíveis. Um modelo generativo de linguagem pode ser empregado numa tarefa discriminativa: basta pedir, em linguagem natural, que classifique um texto em uma de várias categorias, e ele responde com um rótulo. No sentido inverso, o pré-treinamento generativo, em que um modelo apren-

de a prever partes de um texto a partir do restante, costuma ser o ponto de partida de modelos depois ajustados a tarefas discriminativas, como ocorreu com a família BERT. O que muda, ao dizer que um modelo é generativo ou discriminativo, é aquilo para que ele foi treinado e aquilo que consegue produzir, não uma proibição de uso fora dessa caixa. Conhecer o objetivo de origem é o que permite antecipar custo, comportamento e modos de falha, e por isso a distinção segue valendo como bússola.

Exemplo aplicado

Numa linha de inspeção visual, um classificador que decide se uma peça tem ou não defeito é discriminativo: recebe a imagem e devolve um rótulo. Quando defeitos reais são raros e faltam exemplos para treinar esse classificador, um modelo generativo de imagem pode sintetizar fotos plausíveis de peças defeituosas, ampliando o conjunto de treino. Os dois modelos atuam no mesmo problema, mas com objetivos opostos: um decide, o outro produz.

A distinção tem consequências práticas que reaparecem em toda a lente. Modelar a estrutura completa dos dados é uma tarefa mais ampla do que modelar apenas a fronteira entre respostas, e isso costuma cobrar seu preço. Modelos generativos de uso geral tendem a ser maiores, a exigir mais computação para treinar e para operar, e a entregar saídas que precisam de verificação, porque produzir um artefato plausível não é o mesmo que produzir um artefato correto. Um resumo bem escrito pode conter um número errado; um trecho de código sintaticamente perfeito pode não fazer o que se pediu.

Modelos discriminativos enxutos, ao contrário, respondem em microssegundos, com pouca memória, e devolvem uma saída que é, por construção, uma entre poucas opções verificáveis. Essa assimetria entre custo, garantia e necessidade de verificação é o que, mais adiante, sustenta a primeira e mais importante decisão da camada decisória do módulo, entre usar IA Generativa e usar um modelo de ML clássico. Aqui, na camada conceitual, basta reter que a escolha existe e porque existe: porque discriminar e gerar são objetivos diferentes, com custos diferentes.

A assimetria reaparece, por fim, na forma de avaliar o resultado. Um modelo discriminativo tem avaliação relativamente direta: compara-se a resposta prevista com a resposta correta e mede-se o acerto. Um modelo generativo não tem, em geral, uma única resposta certa contra a qual comparar; um bom resumo, uma boa imagem ou um bom trecho de código admitem muitas formas válidas, e julgar a qualidade exige critérios mais elaborados, muitas vezes humanos. Essa dificuldade de avaliação é uma das marcas da lente generativa e uma das principais fontes de risco em projetos que a adotam, porque uma saída convincente pode esconder um erro que nenhuma métrica simples detecta.

Guardar essa dificuldade desde já evita a ilusão de que basta a saída parecer boa para que ela esteja correta.

Resta desfazer uma confusão de vocabulário que atrapalha cedo. A palavra **generativo**, no sentido técnico, designa uma propriedade estatística do modelo, a de aprender a estrutura dos dados e poder amostrar novos exemplos. Ela não é sinônimo de um produto comercial específico nem de uma escala particular. Existem modelos generativos pequenos, treináveis num laptop, e existem modelos discriminativos enormes. Quando o mercado fala em IA Generativa, costuma se referir a uma família específica de modelos generativos de grande escala, os Foundation Models, tratados mais à frente. O conceito, porém, é anterior e mais amplo do que qualquer produto, e mantê-lo separado da marca evita decisões tomadas por moda em vez de por adequação ao problema.

> **Armadilha comum**
>
> A confusão decisória mais cara desta lente é supor que generativo é sempre superior e tratar todo problema como problema de IA Generativa. Decidir se uma peça tem defeito, prever demanda ou ordenar candidatos são perguntas discriminativas, em que um modelo de ML clássico costuma ser mais barato, mais rápido e mais fácil de verificar. Escolher um modelo generativo nesses casos troca uma solução adequada por uma cara e de saída incerta.

Com a distinção entre discriminar e gerar firme, e com a noção de que ela diz respeito ao objetivo de modelagem e ao tipo de artefato produzido, você tem o critério conceitual que organiza toda a lente criativa. A próxima seção usa esse critério para percorrer, em alto nível, as principais famílias de arquitetura que realizam a geração, mostrando que há mais de uma forma de construir um modelo generativo e que cada forma se presta a um tipo de saída.

2. Panorâmica arquitetural em alto nível

Esta seção pertence à camada conceitual e tem por objetivo que você situe uma solução, em alto nível, dentro do panorama arquitetural generativo, reconhecendo a família a que ela pertence sem entrar em detalhe matemático nem de regime de treinamento. A arquitetura é, em termos simples, a forma da função que o modelo realiza. Já se viu na seção anterior que existe mais de uma forma de construir um modelo generativo; aqui essas formas ganham nome. Quatro famílias cobrem a maior parte do que se observa em PD&I industrial: decoder-only, encoder-decoder, difusão e modelos multimodais. Reconhecer a família certa é o que liga o tipo de problema ao tipo de solução.

Um aviso de escopo antes de prosseguir. O mecanismo interno que tornou essas arquiteturas viáveis, a attention, e os regimes de pré-treinamento em escala não serão

abertos aqui; são objeto do curso de LLMs da Fase 2 do programa, e abri-los agora deslocaria o foco da camada conceitual. O que importa neste momento é reconhecer cada família pela forma como recebe a entrada e produz a saída, porque é essa forma, e não a matemática subjacente, que indica para que tipo de problema cada arquitetura serve. A descrição a seguir é deliberadamente esquemática, no nível de quem precisa situar uma solução, não de quem vai implementá-la.

A família hoje dominante na geração de texto e de código é a decoder-only, também chamada de autorregressiva. Ela gera a saída um elemento por vez, da esquerda para a direita, e cada novo elemento é produzido condicionado a tudo o que já foi gerado e ao pedido inicial. O texto cresce como uma bola de neve: a primeira palavra ajuda a decidir a segunda, as duas primeiras ajudam a decidir a terceira, e assim por diante. Essa simplicidade tem um efeito notável. Quase qualquer tarefa de texto pode ser expressa como continuação: responder a uma pergunta, traduzir, resumir, classificar, tudo vira o ato de continuar um texto que descreve a tarefa. É essa generalidade que faz da família decoder-only a base da maioria dos modelos de linguagem atuais, de GPT a Llama e Phi, e o motivo de ela ser, na prática, a porta de entrada da lente generativa.

GLOSSÁRIO

Modelo Autorregressivo

Modelo que gera a saída um elemento por vez, em que cada novo elemento é produzido condicionado a tudo o que já foi gerado. É a base das arquiteturas decoder-only: o texto cresce passo a passo, e cada passo usa os anteriores como contexto.

Dois detalhes de vocabulário acompanham a família autorregressiva e reaparecem na camada aplicada. O primeiro é a noção de janela de contexto, a quantidade de texto que o modelo consegue levar em conta de uma vez ao gerar o próximo elemento; ela limita quanto de um documento cabe num único pedido. O segundo é que o pedido inicial, o prompt, é parte do mesmo texto que o modelo continua, e não um canal separado de instrução: o modelo não distingue, por construção, ordem e conteúdo, apenas continua o texto que recebeu. Essas duas ideias explicam por que formular bem o pedido e respeitar o limite de contexto são preocupações centrais ao usar modelos decoder-only, assunto que a camada aplicada desenvolve.

Uma segunda família, a **encoder-decoder**, separa de forma explícita duas etapas. O encoder lê a entrada inteira e a codifica numa representação interna; o decoder produz a saída condicionada a essa representação. A imagem útil é a de alguém que primeiro lê o documento todo e só então começa a escrever a resposta, em contraste com quem vai escrevendo à medida que lê. Essa separação é vantajosa quando a tarefa é transformar uma sequência inteira em outra: traduzir um documento, resumir um texto longo, converter uma instrução em uma consulta estruturada. Modelos como T5 e BART popularizaram a família. A diferença prática em relação à decoder-only é de ênfase: a encoder-decoder foi desenhada em torno da ideia de ler tudo antes de escrever, o que a torna natural para transformação condicionada, enquanto a decoder-only trata a entrada como o começo de um texto a continuar. Na prática moderna, muitas tarefas de transformação também são resolvidas por modelos decoder-only, o que mostra que a fronteira entre as duas é de conveniência, não de capacidade.

Vale registrar uma família que aparece nas mesmas discussões, mas que não é generativa: os modelos encoder-only, como o próprio BERT em seu uso típico. Eles leem uma entrada e produzem uma representação numérica dela, útil para classificar, buscar ou agrupar, sem gerar um artefato novo. Mencioná-los aqui evita uma confusão comum: nem todo modelo construído com a tecnologia transformer é generativo. O que coloca um modelo no panorama desta seção não é a tecnologia de base, e sim o objetivo de produzir uma saída nova, o critério da seção anterior. Pelo critério, modelos encoder-only pertencem ao território discriminativo, ainda que partilhem a maquinaria com os modelos generativos.

A terceira família, a **difusão**, opera de um modo bem distinto e domina a geração de sinais contínuos, em especial imagens e áudio. Em vez de produzir a saída de uma vez ou elemento a elemento, ela parte de ruído puro e o remove em passos sucessivos, refinando o artefato até que uma imagem coerente emerja. A intuição é a de uma fotografia revelada aos poucos a partir de um chuvisco. A geração pode ser condicionada por um texto, o que permite produzir uma imagem a partir de uma descrição, e esse condicionamento é o que torna a família útil além da arte. Stable Diffusion é o exemplo mais conhecido. Para PD&I industrial, a difusão importa menos pela geração artística e mais pela síntese de dados: gerar imagens plausíveis de uma condição rara para aumentar um conjunto de treino escasso, possibilidade que dialoga diretamente com o uso generativo discutido na seção anterior.

Convém uma nota histórica sobre a geração de imagens. Antes de a difusão se tornar dominante, a abordagem mais conhecida para sintetizar imagens eram as redes generativas adversariais, ou GANs, em que duas redes competem, uma gerando candidatos e outra tentando distingui-los de exemplos reais, de modo que o gerador melhora ao tentar enganar o avaliador. As GANs produziram resultados marcantes, mas são noto-

riamente instáveis de treinar, e em boa parte das aplicações recentes foram substituídas por modelos de difusão, mais estáveis e mais fáceis de condicionar por texto. Para a camada conceitual, basta saber que existe mais de um caminho para gerar imagens e que a escolha entre eles é técnica, não definidora do tipo de saída.

Exemplo aplicado

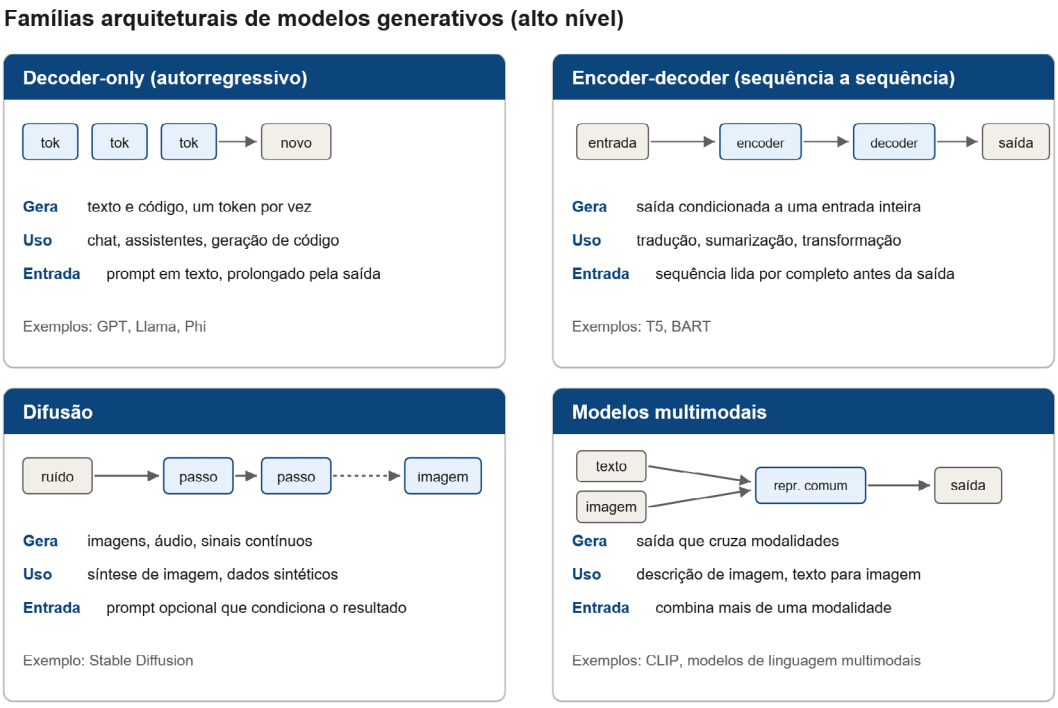
Numa aplicação de inspeção visual industrial, imagens de um defeito raro podem ser escassas demais para treinar um detector confiável. Um modelo de difusão, condicionado por uma descrição, sintetiza imagens plausíveis daquele defeito e equilibra o conjunto de treino. A escolha da família decorre do tipo de saída: como o artefato desejado é uma imagem, a difusão é a candidata natural, e não um modelo de texto.

A quarta família, os **modelos multimodais**, não se define por um mecanismo único, e sim pela capacidade de combinar mais de uma modalidade, como texto e imagem, numa representação comum. A partir dessa representação compartilhada, o modelo pode descrever uma imagem em texto, buscar imagens por uma consulta textual ou gerar uma imagem a partir de uma descrição. Modelos como CLIP estabeleceram a ponte entre texto e imagem, e os modelos de linguagem multimodais recentes incorporam essa ponte ao próprio fluxo de geração, recebendo imagens junto com o texto do pedido. Em PD&I industrial, a multimodalidade é o que permite tratar, num só sistema, um laudo escrito e a imagem que o acompanha, ou uma instrução em texto e a fotografia da peça a que ela se refere.

Um eixo conceitual ajuda a comparar as famílias para além do mecanismo: o que condiciona a geração. Em alguns casos, a geração parte apenas de um pedido curto; em outros, é fortemente condicionada por uma entrada rica, um documento a resumir, uma imagem a descrever, uma instrução detalhada. Decoder-only e encoder-decoder diferem sobretudo em como tratam essa entrada condicionante; a difusão a recebe como orientação do processo de remoção de ruído; os modelos multimodais permitem que o condicionamento venha de mais de uma modalidade ao mesmo tempo. Perguntar o que condiciona a saída costuma ser, na hora de escolher, mais esclarecedor do que perguntar o nome da arquitetura, porque é o condicionamento que decide se a entrada de que você dispõe é suficiente para o artefato que você quer.

A figura a seguir reúne as quatro famílias lado a lado, com o esquema de como cada uma recebe a entrada e produz a saída, além do tipo de artefato gerado e de exemplos representativos.

Figura 1 - Famílias arquiteturais de modelos generativos em alto nível.



Fonte: autor. Link: M1_fig-1_panorama_arquitetural.svg

Para Cego Ver:

Figura intitulada "Famílias arquiteturais de modelos generativos (alto nível)". O conteúdo está organizado em quatro quadros distribuídos em duas linhas e duas colunas, cada um representando uma família de modelos de inteligência artificial generativa. Os títulos aparecem em faixas azuis e os diagramas utilizam caixas conectadas por setas para ilustrar o fluxo de processamento.

No canto superior esquerdo, o quadro "Decoder-only (autoregressivo)" mostra uma sequência de três caixas com a palavra "tok", representando tokens, seguida por uma caixa "novo". As setas indicam que o modelo gera um novo token a partir dos anteriores. O texto informa que esse tipo de modelo gera texto e código, um token por vez, sendo utilizado em chats, assistentes e geração de código. Exemplos citados: GPT, Llama e Phi.

No canto superior direito, o quadro "Encoder-decoder (sequência a sequência)" apresenta o fluxo: entrada, encoder, decoder e saída. O modelo gera uma saída condicionada a uma entrada completa e é utilizado em tarefas como tradução, sumarização e transformação de textos. Exemplos: T5 e BART.

No canto inferior esquerdo, o quadro "Difusão" apresenta uma sequência composta por ruído, passo, passo e imagem. O diagrama representa a transformação gradual de ruído em uma imagem final. O texto informa que essa arquitetura gera imagens, áudio e outros sinais contínuos, sendo aplicada em síntese de imagens e geração de dados sintéticos. Exemplo: Stable Diffusion.

No canto inferior direito, o quadro "Modelos multimodais" mostra duas entradas, texto e imagem, convergindo para uma representação comum e, em seguida, para uma saída. O modelo combina diferentes modalidades de dados e pode realizar tarefas como descrição de imagens ou geração de imagens a partir de texto. Exemplos citados: CLIP e modelos de linguagem multimodais.

Na parte inferior da figura aparece a indicação de fonte, atribuída ao autor, acompanhada de um link para o arquivo da imagem. A figura tem como objetivo comparar, de forma resumida, as principais arquiteturas utilizadas em sistemas de inteligência artificial generativa.

A figura organiza as quatro famílias em cartões. Decoder-only gera texto e código token a token; encoder-decoder lê a entrada inteira antes de produzir a saída; difusão parte de ruído e o remove em passos sucessivos para formar imagens e áudio; modelos multimodais combinam modalidades numa representação comum. Em alto nível, todas geram artefatos e diferem sobretudo no tipo de saída e em como a entrada condiciona a geração.

Lida a figura, o uso prático do panorama é direto. Diante de uma necessidade, identifica-se primeiro o tipo de artefato desejado e, a partir dele, a família candidata: texto ou código apontam para decoder-only; a transformação de um texto inteiro em outro sugere encoder-decoder, embora decoder-only também resolva; imagem ou áudio apontam para difusão; a presença de mais de uma modalidade no problema aponta para um modelo multimodal. Esse mapeamento não esgota a decisão, que envolve custo, disponibilidade e qualidade exigida, mas dá o primeiro passo e evita perguntas mal colocadas, como buscar um modelo de texto para um problema cuja saída é uma imagem. Situar a solução na família certa é, em si, parte do que a competência desta seção pede.

Duas observações fecham o panorama. A primeira é que, em alto nível, todas as quatro famílias geram artefatos; o que as distingue é o tipo de saída e o modo como a entrada condiciona a geração, e não uma hierarquia de qualidade entre elas. A segunda é que as fronteiras entre famílias borram na prática: modelos decoder-only recentes incorporam entradas de imagem e se tornam multimodais, e técnicas de difusão e de autorregressão convivem em sistemas híbridos. Por isso o nome da família é menos importante do que a pergunta que ele ajuda a responder: que tipo de artefato a solução precisa produzir, e a partir de que entrada. Reconhecer a família certa é um atalho para antecipar custo, formato de saída e adequação, não um fim em si mesmo.

> **Para aprofundar**
>
> O mecanismo de attention, que sustenta a arquitetura transformer comum a estas famílias, e os regimes de pré-treinamento em escala são tratados no curso de LLMs da Fase 2 do programa. A referência fundadora é VASWANI, A. et al. Attention Is All You Need. NeurIPS, 2017, disponível em arxiv.org/abs/1706.03762.

Reconhecidas as famílias e o tipo de saída a que cada uma se presta, falta um último elemento conceitual para completar a camada: entender por que alguns desses modelos, ao crescerem em escala e em generalidade, passaram a servir de base reaproveitável para muitas tarefas, em vez de serem treinados do zero para cada uma. É o tema da próxima seção, sobre Foundation Models.

3. Foundation Models e capacidade emergente

Esta seção pertence à camada conceitual e tem por objetivo que você compreenda o que torna um modelo um Foundation Model e o que a expressão capacidade emergente significa para o uso prático. O termo Foundation Model já foi apresentado no M0, na seção 0.2, como um modelo de grande escala treinado em dados gerais que serve de base para muitas tarefas mediante adaptação leve. Aqui não se redefine o termo; aprofundam-se os três traços que o caracterizam e situa-se a propriedade que mais gera mal-entendidos, a emergência, em seu devido lugar.

Três traços convivem num Foundation Model. O primeiro é a escala: da ordem de centenas de milhões a centenas de bilhões de parâmetros, treinados sobre volumes enormes de dados. O segundo é a generalidade do dado de treino: o modelo não é treinado para uma tarefa específica, e sim sobre um corpus amplo que cobre muitos domínios, o que lhe dá um repertório geral em vez de uma especialidade estreita. O terceiro, e o mais importante para a prática, é a possibilidade de adaptação leve: em vez de treinar um modelo do zero para cada tarefa, parte-se de um modelo já treinado e o ajusta com pou-

co esforço. É esse terceiro traço que muda a economia de um projeto, porque desloca o grosso do custo do treinamento, caro e único, para a adaptação, comparativamente barata e repetível. Sem ele, a escala seria apenas uma despesa; com ele, a escala vira um ativo reaproveitável.

Convém detalhar o primeiro traço, a escala, porque ele tem mais de uma dimensão. Crescer um modelo significa aumentar, de forma conjunta, três grandezas: o número de parâmetros, o volume de dados de treino e a quantidade de computação empregada. Observou-se, ao longo da última década, que o desempenho desses modelos melhora de maneira relativamente previsível à medida que essas grandezas crescem juntas, regularidade que ficou conhecida como leis de escala. Foi essa previsibilidade que transformou a escala de uma aposta numa estratégia: investir em modelos maiores passou a ser uma forma esperada de obter modelos mais capazes. Entender a escala como estratégia, e não como mágica, é o que prepara a leitura crítica da emergência mais adiante.

O segundo traço, a generalidade do dado, está ligado ao modo como esses modelos são treinados. Treiná-los sobre um corpus amplo só é viável porque o aprendizado é, em larga medida, auto-supervisionado: o modelo aprende a prever partes do dado a partir de outras, sem depender de rótulos fornecidos por pessoas. Como visto na seção 1.1, isso libera o treino da dependência de dados rotulados, escassos e caros, e permite usar texto, imagem e código disponíveis em grande quantidade. A generalidade do repertório resultante é o que torna a mesma base útil para tarefas muito diferentes, e é a condição para o terceiro traço, a adaptação leve, fazer sentido econômico.

Um exemplo torna a generalidade concreta. A mesma base pode, sem novo treinamento, ajudar a redigir um e-mail, esboçar um trecho de código, resumir um relatório e responder a uma pergunta sobre um procedimento, tarefas que, na prática anterior, exigiriam modelos distintos, cada um com seu conjunto de dados e seu ciclo de treino. Isso não significa que a base faça todas igualmente bem, nem que dispense verificação; significa que um único ativo cobre um leque amplo de necessidades com adaptação leve. É essa cobertura ampla a partir de um só investimento de treino que explica o interesse econômico pelos Foundation Models.

A adaptação leve assume três formas principais, que a camada aplicada deste módulo trata em detalhe e que aqui apenas nomeamos. No prompting, ou aprendizado em contexto, o modelo recebe uma instrução em linguagem natural, possivelmente com alguns exemplos, e produz a saída sem nenhum treinamento adicional. No fine-tuning leve, continua-se o treinamento do modelo sobre dados específicos da tarefa, ajustando apenas uma fração pequena dos parâmetros para baratear o processo. Na recuperação de contexto, não se altera o modelo: injeta-se, no próprio pedido, o conteúdo externo de que a resposta depende. O que todas compartilham é não repetir o pré-treinamen-

to: aproveitam o repertório geral já existente e o especializam. A escolha entre essas formas é, ela própria, uma decisão de engenharia, retomada na camada decisória; o quadro a seguir as resume.

Quadro 2 - Formas de adaptar um Foundation Model.

Forma	O que ajusta	Quando tende a caber
Prompting (em contexto)	nada; usa instrução e exemplos no próprio pedido	tarefas cobertas pelo repertório geral; prototipagem rápida
Fine-tuning leve	uma fração pequena dos parâmetros	tarefa recorrente, com dados próprios e formato específico
Recuperação de contexto	nada no modelo; injeta documentos no pedido	quando a resposta depende de conteúdo externo e atualizável

Para Cego Ver:

Quadro intitulado "Quadro 2 - Formas de adaptar um Foundation Model". A imagem apresenta uma tabela organizada em três colunas: Forma, O que ajusta e Quando tende a caber.

Na primeira linha, referente a Prompting (em contexto), a tabela informa que não há ajuste no modelo. Em vez disso, utilizam-se instruções e exemplos diretamente no pedido. Essa abordagem é indicada para tarefas já contempladas pelo repertório geral do modelo e para prototipagem rápida.

Na segunda parte da tabela, após um espaço visual contendo o logotipo da Industr.IA, aparecem mais duas formas de adaptação.

A linha Fine-tuning leve descreve que uma pequena fração dos parâmetros do modelo é ajustada. Essa estratégia tende a ser adequada para tarefas recorrentes, que utilizam dados próprios e exigem formatos específicos de resposta.

A linha Recuperação de contexto informa que não há alteração no modelo; documentos externos são inseridos no contexto da solicitação. Essa abordagem é recomendada quando a resposta depende de conteúdo externo, atualizado ou específico do domínio.

O quadro tem como objetivo comparar diferentes estratégias para adaptar um Foundation Model, destacando o que é modificado em cada abordagem e em quais situações seu uso é mais apropriado. Além disso, há sublinhados vermelhos sob várias palavras, indicando possíveis marcações de revisão ortográfica do editor de texto.

Há ainda uma etapa de adaptação que merece nota, porque distingue um modelo de base cru de um modelo pronto para uso. Um Foundation Model treinado apenas para prever a continuação de textos não segue instruções de forma natural; ele continua o texto, mas não necessariamente faz o que se pede. Para torná-lo útil como assistente, aplica-se uma etapa adicional, em que o modelo é ajustado para seguir instruções e para alinhar suas respostas a preferências humanas. Os métodos dessa etapa pertencem ao curso de LLMs da Fase 2, mas a ideia importa aqui: o modelo que se usa na prática raramente é a base crua, e sim uma versão já adaptada a seguir instruções. Isso ajuda a explicar por que esses modelos respondem tão bem a pedidos em linguagem natural, e por que dois modelos da mesma família podem se comportar de modo distinto conforme a adaptação que receberam.

Resta o traço que mais gera confusão. À medida que esses modelos crescem em escala, observa-se que certas habilidades aparecem de forma relativamente abrupta a partir de um certo tamanho, sem terem sido programadas e sem estarem presentes em modelos menores. Esse fenômeno recebeu o nome de capacidade emergente. Exemplos relatados incluem resolver problemas aritméticos de vários passos ou seguir instruções em formatos não vistos durante o treino, habilidades que modelos pequenos não exibem e que surgem em modelos grandes. A observação alimentou boa parte do entusiasmo recente com a escala e merece ser entendida com cuidado, porque é facilmente

exagerada. Um caso particularmente citado é o próprio aprendizado em contexto, a capacidade de um modelo grande resolver uma tarefa nova a partir de poucos exemplos colocados no pedido, sem qualquer treinamento adicional; em modelos pequenos esse comportamento é fraco ou ausente, e em modelos grandes ele se torna utilizável, o que o tornou um dos exemplos mais discutidos de habilidade associada à escala.

GLOSSÁRIO

Capacidade emergente

Habilidade que aparece em modelos de grande escala e não está presente em modelos menores, sem ter sido programada de forma explícita. O termo descreve a observação de que certas habilidades surgem com o aumento de escala; parte da aparência de salto abrupto, porém, depende da métrica usada para medir a habilidade.

A emergência exige cautela interpretativa, e aqui está um ponto que separa leitura técnica de leitura de marketing. Trabalhos recentes mostraram que parte do que se descreve como emergência abrupta pode ser, em boa medida, um efeito da métrica escolhida para medir a habilidade: métricas que só pontuam acertos perfeitos criam a impressão de um salto, enquanto métricas mais graduais revelam uma melhora contínua à medida que a escala aumenta. Isso não nega que modelos maiores fazem mais; nega que haja necessariamente um limiar mágico a partir do qual habilidades surgem do nada. Para a prática, a lição é dupla: a escala traz ganhos reais, mas a palavra emergência não deve ser tomada como promessa de que basta crescer o modelo para obter, de forma confiável, qualquer habilidade desejada. Decisões de projeto fundadas em emergência prometida, e não em avaliação medida, são decisões frágeis.

O valor dos Foundation Models para PD&I industrial vem justamente da combinação de generalidade com adaptação leve. Reaproveitar uma base ampla reduz a quantidade de dados rotulados necessária por tarefa, acelera a prototipagem e permite atacar problemas para os quais não haveria dados suficientes para treinar um modelo do zero. Esse ganho, no entanto, vem com contrapartidas que a camada decisória trata em detalhe: o custo de inferência de um modelo grande, a necessidade de verificar saídas que são plausíveis mas não garantidas, e a dependência de um modelo cujo comportamento não foi desenhado para a sua tarefa específica. Adotar um Foundation Model é uma escolha de engenharia com custos próprios, não um caminho sem atrito.

Exemplo aplicado

Uma equipe precisa responder a perguntas sobre a documentação técnica interna de um equipamento. Em vez de treinar um modelo do zero, que exigiria muitos exemplos rotulados, adapta um Foundation Model por prompting, fornecendo a instrução e trechos da documentação no próprio pedido. O esforço migra da coleta de dados de treino para a engenharia do pedido e a verificação das respostas.

A consequência mais ampla é uma mudança no modo de trabalhar. Durante anos, a prática padrão era treinar um modelo específico para cada tarefa, a partir dos próprios dados. Com Foundation Models, a prática se desloca para adaptar uma base compartilhada, e o esforço migra da coleta massiva de dados rotulados para a engenharia da adaptação e, sobretudo, para a avaliação das saídas. Isso reorganiza onde está o risco de um projeto: menos na obtenção de dados de treino, mais na verificação de que o modelo adaptado faz o que se espera, de forma confiável e a um custo aceitável. Quem entende essa migração de risco já está em posição de tomar as decisões que o restante do módulo formaliza.

Essa migração do risco para a avaliação tem um desdobramento que o restante do curso desenvolve. Se o que mais importa deixou de ser o dado de treino e passou a ser a verificação da saída, então definir como medir a qualidade e a segurança das respostas torna-se parte central do projeto, e não um detalhe final. Em PD&I industrial, isso significa tratar a avaliação como entregável, com critérios explícitos sobre o que conta como resposta aceitável, em vez de confiar na impressão de que a saída parece boa. A lente conceitual apenas aponta esse deslocamento; a camada decisória o transforma em eixos concretos de decisão, e a camada aplicada mostra onde, no desenho de uma solução, a verificação precisa entrar.

Armadilha comum

Outra confusão decisória frequente é supor que adotar um Foundation Model dispensa a decisão entre IA Generativa e ML clássico, ou que a escala do modelo garante correção. Um modelo grande produz saídas plausíveis, não necessariamente corretas, e continua exigindo verificação. A adoção de uma base ampla é uma escolha com custo de inferência e de avaliação, não um atalho que encerra as decisões do projeto.

Com os três traços de um Foundation Model claros, com as formas de adaptação leve nomeadas e com a emergência situada em seu devido lugar, fecha-se o vocabulário conceitual da lente criativa. Os três modos de adaptação apenas nomeados aqui,

prompting, recuperação de contexto e fine-tuning leve, são o objeto da camada aplicada, que ensina a reconhecê-los numa especificação de solução, com seus componentes mínimos e seus pontos de ruptura.

Conclusão

As três seções deste texto-base cobriram, em conjunto, o vocabulário conceitual da lente criativa: a distinção entre modelar para decidir e modelar para produzir, isto é, discriminativo versus generativo; as quatro famílias de arquitetura que realizam a geração em alto nível, decoder-only, encoder-decoder, difusão e modelos multimodais; e o que torna um modelo um Foundation Model, incluindo o sentido preciso de capacidade emergente. Cada bloco foi tratado em alto nível, sem matemática nem implementação, com foco em nomear corretamente e reconhecer famílias.

O ganho prático desse vocabulário é a capacidade de situar uma solução antes de escolhê-la. Diante de um problema, saber dizer se ele pede um artefato novo ou uma decisão entre opções já indica se a lente generativa é sequer pertinente; reconhecer o tipo de saída desejado, texto, imagem ou combinações multimodais, aponta a família arquitetural natural; e identificar que uma base ampla pode ser adaptada, em vez de treinada do zero, antecipa onde estarão os custos do projeto. Esse posicionamento é o que evita a decisão por moda, e é a matéria-prima das camadas seguintes.

A camada aplicada, que vem a seguir, parte exatamente deste ponto. Os três modos de adaptação de um Foundation Model apenas nomeados aqui, prompting estruturado, recuperação de contexto e fine-tuning leve, tornam-se ali padrões a reconhecer numa especificação de solução, com seus componentes mínimos e seus pontos de ruptura. Reconhecer qual padrão está em jogo é o que liga o vocabulário conceitual à prática de PD&I industrial.

Da camada conceitual sai também a pergunta que organiza a camada decisória do módulo. Como discriminar e gerar são objetivos diferentes, com custos e garantias diferentes, a primeira decisão de qualquer solução candidata é entre usar IA Generativa e usar um modelo de ML clássico para a tarefa em jogo. Esta seção forneceu o critério dessa decisão; a camada decisória fornecerá os eixos para tomá-la e defendê-la diante de quem não é da área técnica, em termos de cronograma, custo e validação.

Vale lembrar, por fim, que a IA Generativa é tratada aqui como lente, e não como tecnologia paralela. O vocabulário desta seção não descreve um nicho separado do restante do curso: ele oferece um ângulo de leitura aplicável a qualquer problema que admita a síntese de um artefato, e é por esse ângulo que os próximos objetos do módulo conduzirão a discussão.

REFERÊNCIAS

Bibliografia mínima

VASWANI, A. et al. Attention Is All You Need. In: Advances in Neural Information Processing Systems (NeurIPS), 2017. Preprint em https://arxiv.org/abs/1706.03762.

BOMMASANI, R. et al. On the Opportunities and Risks of Foundation Models. Center for Research on Foundation Models, Stanford, 2021. Preprint em https://arxiv.org/abs/2108.07258.

HO, J.; JAIN, A.; ABBEEL, P. Denoising Diffusion Probabilistic Models. In: Advances in Neural Information Processing Systems (NeurIPS), 2020. Preprint em https://arxiv.org/abs/2006.11239.

Bibliografia complementar

DEVLIN, J. et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In: NAACL, 2019. Preprint em https://arxiv.org/abs/1810.04805.

WEI, J. et al. Emergent Abilities of Large Language Models. Transactions on Machine Learning Research (TMLR), 2022. Preprint em https://arxiv.org/abs/2206.07682.

SCHAEFFER, R.; MIRANDA, B.; KOYEJO, S. Are Emergent Abilities of Large Language Models a Mirage? In: Advances in Neural Information Processing Systems (NeurIPS), 2023. Preprint em https://arxiv.org/abs/2304.15004.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. Deep Learning. Cambridge: MIT Press, 2016. Capítulo 20, Deep Generative Models. Disponível em https://www.deeplearningbook.org.

Industr.IA

Residência em TIC 49



Executor iRede

Executor Atlântico

Coordenador Softex

Financiador MCTI Futuro

Ministério da Ciência, Tecnologia e Inovação

Governo do Brasil — Do lado do povo brasileiro

Projeto apoiado pelo Ministério da Ciência, Tecnologia e Inovações, com recursos da Lei nº 8.248, de 23 de outubro de 1991.